

Metadata

[personal project](#)

- **Author:** Connor Ng
- **Status:** Draft
- **Last updated:** 2026-08-11

Overview

A mobile-friendly app that replaces my spreadsheet + separate logging app with one tool: test a lift, get an auto-generated 5-week percentage-based program, log sets against it, and let the app learn a personal correction factor over time so the numbers get more accurate the longer I use it.

Problem Statement

I run 5-week strength training blocks and recalculate my working weights each block off a tested 1-rep max (E1RM). That calculation currently lives in a spreadsheet, and logging actual sessions happens in a separate app — two disconnected tools for one workflow. The spreadsheet step is manual and easy to skip, which is the actual source of my inconsistency. Existing fitness apps don't solve this: they're either generic loggers with no programming logic, or they use a static formula (Epley) that doesn't account for how that formula's accuracy degrades for trained lifters and at higher rep counts — something I hit directly during scoping (see Alternatives Considered).

Target User

Me, daily, real usage at the gym — not a demo. Needs to be fast enough to log mid-workout without friction.

Goals

- Enter a test lift and get an E1RM calculated automatically
- Auto-generate the block's week-by-week training weights off that E1RM — no manual spreadsheet math
- Log actual sets/reps/weight against those targets
- Track a cut: bodyweight and calories/macros
- Track daily steps
- Learn a personal correction factor over time instead of trusting a generic formula indefinitely

- Everything in one place, on my phone, with none of the bloat of a general fitness app

Non-Goals (v1)

- Food/barcode database — manual macro entry is enough to start
- Multi-user / accounts for anyone but me
- Native App Store app — a PWA avoids that overhead entirely
- Auto step import from Health/Fit APIs — manual entry first, revisit later
- A library of pre-built programs for other people — this is about *my* block structure, not templates for others
- Squat and OHP tracking — not part of current training focus; the data model isn't lift-limited, so this is a config change later, not a redesign

Alternatives Considered

- **An existing Obsidian-style "chat with your data" pattern doesn't apply here, but the equivalent check does:** researched existing AI-assisted fitness trackers (several exist, including ones built with Claude Code specifically) — none combine percentage-based block programming with a continuously self-correcting calibration loop. Most either log freely with no programming logic, or use a static formula with no correction mechanism. Building this is the point: the correction-factor logic is the differentiated and highest-learning-value part of the project.
- **Raw Epley formula alone, no calibration:** rejected after directly hitting its failure mode during scoping — a real RPE-based test (225 @ RPE9) was reliable, but reasoning through a hypothetical high-rep test showed Epley overestimating badly outside low rep ranges, which is a known limitation for trained lifters. A single static formula isn't trustworthy enough to build a whole program around.
- **Self-hosted backend (e.g. home server + Tailscale):** rejected for v1 — more setup than a hosted free-tier backend, and doesn't work from arbitrary networks (gym wifi/cellular) without extra configuration.
- **Native app via App Store:** rejected — unnecessary review/distribution overhead for a single-user tool; a PWA installs to the home screen with the same day-to-day experience.

User Stories

- As the lifter, I want to enter a test result and have the app generate my full week-by-week weights for the block, so I stop doing that math in a spreadsheet.
- As the lifter, I want to choose how a test is interpreted (a straight rep test, an RPE-based estimate, or my own judgment call) depending on what I actually did that day, so the app doesn't force one formula onto every kind of test.

- As the lifter, I want to flag a set as a genuine max effort, so the app can compare its prediction to reality and adjust its correction factor for that lift going forward.
- As the lifter, I want to log body weight, macros, and steps in the same app, so I'm not tracking a cut somewhere else entirely.
- As the lifter, I want to open this on my phone at the gym and log a set in a few seconds, so it doesn't get skipped mid-session.

Requirements

Must Have (P0)

Minimum to test the core hypothesis: does auto-generated programming actually replace the spreadsheet.

- ☐ Program model — create a 5-week block with a start date
- ☐ TestLift entry with four modes: raw test (Epley), RPE-based, manual E1RM, or manual Week-1 weight — chosen based on what kind of test was actually performed
- ☐ Auto-generate Week 1 (percentage of E1RM) and Weeks 2–5 (cumulative additive increments, configurable per lift) — all values rounded to the smallest loadable plate increment (5 lb, confirmed against actual available plates)
- ☐ Structured logging UI for sets (weight, reps, optional RPE, optional max-effort flag) — no natural language yet
- ☐ Basic dashboard: current week, target vs. actual per lift
- ☐ Deployed as an installable PWA, usable on phone at the gym

Should Have (P1)

Only worth building once P0 proves the core loop gets used daily.

- ☐ Natural-language logging via Claude API (e.g. "bench 3x8 at 185")
- ☐ Auto-calibration: compare max-effort set outcomes to predictions, update a per-lift correction factor via exponential moving average, gated by a minimum data-point guardrail
- ☐ Body metrics (weight, calories, macros) + cut-tracking view
- ☐ Step tracking (manual entry)
- ☐ Block-over-block comparison view

Nice to Have (P2)

- ☐ Weekly coaching-style summary (e.g. flag a stalled lift)
- ☐ Auto step import from Health/Fit APIs
- ☐ Expand lift list back to Squat/OHP once core lifts are stable

Success Metrics

Functional

- Auto-calculated E1RM and weekly targets match the original spreadsheet formula exactly (verified: Squat $300 \times 1 \rightarrow 310$, Bench $225 \times 1 \rightarrow 230$, OHP $135 \times 1 \rightarrow 140$, Deadlift $375 \times 1 \rightarrow 385$)
- I stop opening the spreadsheet — this is the real test
- I complete one full 5-week block start to finish using only the app
- (P1) Natural-language logging parses correctly without correction most of the time — not measurable until P1 ships

Learning

- I can explain why Program/TestLift/WeeklyTarget/Calibration are separate tables rather than one flat structure
- I've hit and personally debugged at least one real failure (a bad parse, a wrong calculation, a miscalibrated correction factor)

Design / Mockups

(To do: rough wireframe of the mobile logging flow and dashboard before starting the P0 build)

Open Questions

- ☐ Week 1 starting percentage — confirmed at 0.75 for Bench (backed into 205 as a manual Week-1 entry); Deadlift and Weighted Pull-up still need their own test data before their formulas are locked
- ☐ Deadlift and Weighted Pull-up currently use a placeholder increment sequence ([+10, +10, +5, +5]) copied from Bench's confirmed real data — needs its own real test data to confirm or correct
- ☐ Calibration EMA alpha value — a reasonable starting point (~ 0.3 – 0.4) is chosen, but won't be tunable with real judgment until enough max-effort data points accumulate
- ☐ With only ~ 1 – 2 max-effort data points per lift per 5-week block, calibration converges slowly — worth deciding whether it ships in P1 as designed (quietly accumulating data) or is deferred until a few blocks of history already exist

Timeline

Milestone	Date
Spec finalized	

Milestone	Date
P0 (data model, manual logging, dashboard, PWA deploy) complete	
P1 (NL logging, calibration, body metrics, steps) complete	
One full block completed using only the app	
Retrospective write-up	

Changelog

- **2026-08-11:** Drafted from scoping conversation — settled on lift scope (Bench/Weighted Pull-up/Deadlift, not all four), confirmed exact formulas against real spreadsheet data, designed the calibration mechanism and its guardrails, resolved the plate-resolution rounding constraint, and worked Bench through as a fully closed real example.